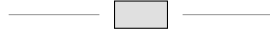


---

# THE SANDBOX

*Cellular Automata and Version Control*



SEDS CELESTIA · SIMULATIONS

INDUCTIONS 2026 · WEEK 2

---

**Due:** EOD, 23rd September 2026

**Prerequisites:** Week 1 (rigid balls), basic NumPy

---

This assignment is arranged in tasks of increasing difficulty. You are expected to attempt all of them and submit the Google Form with your GitHub repository linked. Partial submissions are read: genuine effort and clear thinking count for more than a finished notebook that you cannot explain. **Along with the code, write a short assignment brief in your repository's README.md: answer the two questions marked in the text, and say what you make of your own results. Half a page is plenty; this should not take you long.**

**Introduction.** Last week you simulated rigid balls: continuous positions, Newton's equations, collision detection between distinct bodies. This week the world changes shape. Instead of a handful of objects moving through continuous space, you get a grid of cells, each one holding a material that decides what to do by looking only at its immediate neighbours. No velocities, no forces, no collisions in the Newtonian sense, just local rules applied everywhere at once.

This kind of model is called a *cellular automaton*. The name is unfamiliar but the idea is not: Conway's Game of Life is one, and so is every "falling sand" game you have seen online. The striking thing about all of them is how much structure appears from how little information. A sand grain has no idea it is part of a dune; it only knows whether the cell below it is empty. A water molecule does not solve the Navier–Stokes equations; it just slides sideways when it cannot fall. Yet pour enough of them onto a grid and you get pooling, piling, and flow that looks, if you squint, like physics.

You will build a two-dimensional falling-sand simulator in Python, drawn with Pygame. The starter template gives you the grid, the rendering, and the mouse input. Your job is to write the physics: the update rules that make sand fall and water spread. Once the basics work, you will extend the world with fire, smoke, and wood as optional bonus materials.

A word on version control. This assignment is longer than last week's, and you will

be modifying code that already works before you touch the physics. Try to use git and explore it's features as you do the assignment. Keep making commits to save your progress with proper commit messages.

## A NOTE ON NOTATION

---

Two conventions carry over from Week 1, and one new one appears.

**Grid coordinates.** The world is a rectangular array of cells, indexed by row  $y$  and column  $x$ . Row 0 is the *top* of the screen; row  $H-1$  is the bottom. Column 0 is the left edge; column  $W-1$  is the right. A cell's contents are written  $G(y, x)$ , where  $G$  is the grid and the pair  $(y, x)$  picks out one entry. In NumPy this is just `grid[y, x]`.

**The left arrow is assignment, not equality.**

$$a \leftarrow f(a) \quad \text{means} \quad a_{\text{new}} = f(a_{\text{old}}), \quad \text{in Python: } \mathbf{a} = \mathbf{f(a)}.$$

This is the same convention as Week 1. Every rule below is written as an assignment: compute the right-hand side from the current state, then store the result.

**The swap grid.** A second array, written  $G'$ , starts each tick as a copy of  $G$ . Rules read from  $G$  and write into  $G'$ . At the end of the tick the two arrays exchange roles. This double-buffering is explained in detail in Section I; the notation exists so that the rules can be written compactly without losing track of which grid is being read and which is being written.

## I. THE UPDATE RULES

---

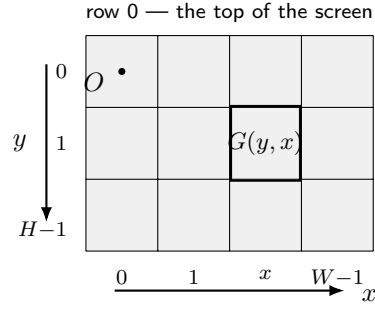
**The grid as the world.** Each cell stores exactly one material, represented as a small integer. The `Material` enum in the starter code assigns names to these integers:

```
EMPTY = 0
SAND  = 1
WATER = 2
```

Empty is zero on purpose: NumPy fills a new array with zeros, so an empty world is free.

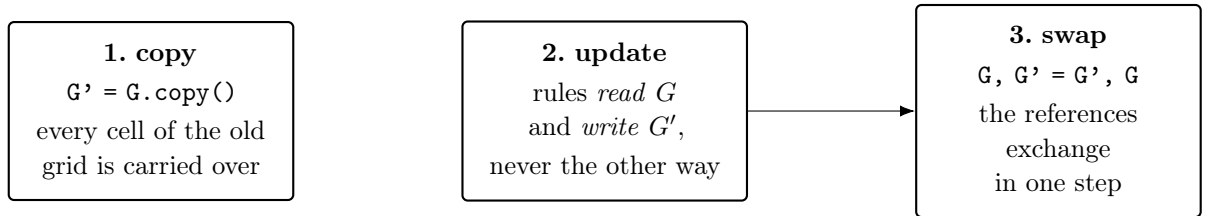
**One tick, one pass.** Each tick, every non-empty cell in the grid is visited once and asked to apply its rule. The rule reads the neighbours in the current grid and writes the result into the swap grid. A cell that moves claims its destination in  $G'$ ; a cell that stays put is already there (because  $G'$  started as a copy of  $G$ ).

The ordering of the visit matters. Rows are scanned **bottom-to-top**, starting at  $y = H-1$  and ending at  $y = 0$ . This ensures that when a grain is processed, the cell below it has already been decided: if it moved out of the way, the current grain can fall into the space it left; if it stayed, the current grain is blocked. Processing top-to-bottom would let a single grain fall several cells in one tick, which looks like flickering.



**Figure 1.** The grid and its coordinates.  $G(y, x)$  is the material stored at row  $y$ , column  $x$ . Row 0 is the top of the screen and rows increase downward, which is exactly the ordering NumPy uses in `grid[y, x]`: the first index counts down the rows.

Within a row, columns are visited in a **fresh random order** each tick. The permutation is generated once at the start of the tick with `numpy.random.permutation(W)`. This cancels left-right bias: without it, diagonal slides would always prefer one side and every pile would lean.



**Figure 2.** The double-buffered tick. First  $G'$  copies  $G$ , so every cell that does not move this tick survives unchanged. Then each rule reads from the settled  $G$  and writes its result into  $G'$ . Last, the two grids swap roles by exchanging references, which is one algebraically cheap line.

**Question 1.** Why does the swap grid start as a copy of the current state, rather than being filled with zeros? What would happen to a grain that does not move if  $G'$  started empty?

**Part A. Sand.** Sand is the simplest material. Its rule has three cases, checked in order. Let  $(x, y)$  be the current cell and  $G'$  the swap grid.

1. **Fall.** If the cell directly below is empty in  $G'$ , move down:

$$G'(y+1, x) \leftarrow G(y, x), \quad G'(y, x) \leftarrow 0.$$

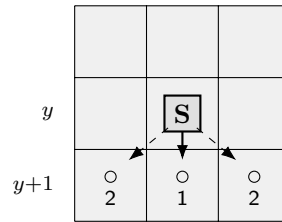
The check is against  $G'$ , not  $G$ : a cell below that already moved this tick has left its destination empty, and we should fall into it.

2. **Slide.** If the cell below is occupied, try the two diagonal-below cells. Pick one at random:

$$\text{if } G'(y+1, x-1) = 0 \text{ and } G'(y+1, x+1) = 0 \text{ then choose randomly.}$$

If only one diagonal is free, take it. If neither is free, the grain stays where it is. This diagonal slide is what gives sand piles their natural slope.

3. **Rest.** If neither case applies, do nothing. The grain is at rest.



**Figure 3.** Sand movement. The grain at  $(x, y)$  first tries the cell straight below it (choice 1); if that cell is occupied it tries the two diagonal-down cells (choice 2), picking one at random or taking the single free one. The numbers are a *priority*, not a schedule: the diagonals exist only because the straight drop failed.

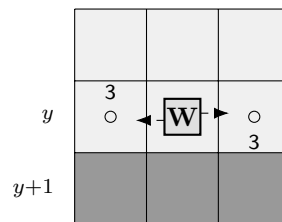
The order - straight down first, then diagonal - is what distinguishes a pile from a fluid. Sand that can only fall straight down stacks in vertical columns. Sand that slides diagonally but only after failing to fall straight down builds a  $\sim 45^\circ$  slope, which is close to the angle of repose in real granular materials.

**Part B. Water.** Water starts with the same two rules as sand: fall if you can, slide diagonally if you cannot. The third case is new:

3. **Spread.** If the grain could neither fall nor slide, and it is a liquid, try to move sideways into a random empty horizontal neighbour:

if  $G'(y, x-1) = 0$  or  $G'(y, x+1) = 0$  then choose one at random.

This sideways flow is what makes water pool flat and fill containers. Sand never does this, which is why sand piles up and water does not.



**Figure 4.** Water spread. The grey row shows what happens when the cell below and both diagonals are occupied: the drop cannot fall at all. A liquid then leaks sideways into a random empty neighbour on its own row (choice 3). Sand has no such rule, and that single difference separates a pile of sand from a puddle of water.

Implement sand first. Get a single stream of grains falling and piling correctly. Then add water and watch it spread. Both should be working before you move on.

**DELIVERABLE.** A Python file containing your `SandSim` class with the `update()` method implemented for sand and water. The starter template provides everything else: the grid, the rendering, the mouse input. Run it with `uv run python assignment/temp.py`.

**Question 2.** Remove the randomised column order and replace it with a fixed left-to-right scan. Run the simulation for a few hundred ticks. What happens to the shape of a sand pile? Why?

## II. BONUS

OPTIONAL. Everything in this section is optional. Attempt it only once the sand and water of Section I actually work, and treat anything you manage here as a bonus on top of a finished assignment, never as a substitute for one.

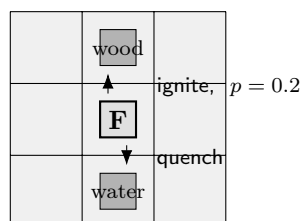
Sand and water never stop being what they are. A grain of sand stays a grain of sand for its whole life; it just moves, and the material in a cell never changes. Fire breaks that rule. It appears, burns, reacts with whatever touches it, and dies. Two ideas make that possible, and both are new.

**Lifetime.** Every fire and smoke cell carries a private age, stored not in the grid but in a second NumPy array (`_attrs`). Each tick the age goes up by one. On crossing a material-specific limit—60 ticks for fire, 40 for smoke—the cell reverts to empty. This is the only per-cell state in the whole simulation; everything else is decided by the material and its neighbours.

**Fire.** Fire rises like smoke and leaks sideways, but its reason to exist is what it does to the eight cells around it:

- If a neighbour is **wood**, it may catch: with probability 0.2 per tick that wood cell becomes fire, age reset to zero.
- If a neighbour is **water**, *this* fire cell is quenched: it becomes smoke, age reset to zero.

Always-on ignition would march across the wood in a perfect square, a flame front with right-angled corners. At  $p = 0.2$  the front goes jagged instead, and the ragged edge is what sells it as burning rather than filling.



**Figure 5.** Fire’s two reactions with its eight neighbours. Wood *may* ignite: with probability  $p = 0.2$  per tick the wood cell becomes fire. Water always wins: a fire cell touching water turns to smoke. Both writes land in the swap grid  $G'$  with a fresh age of zero.

**Smoke.** Smoke rises, and that is about it. It tries to move up one row, then sideways, and it fades when its age runs out. It has no reactions of its own—smoke does not set

things alight or put them out—so this is fire’s movement code with the reaction block removed.

**Wood.** Wood never moves and never ages. It sits in the grid until something else changes it, and fire is the only something that does. That is the point of it: wood exists to be an obstacle and a fuel, and the interest lives in the reaction, not in the wood.

Adding a material is always the same three steps: a new entry in the `Material` enum, a colour in `PALETTE`, and a rule in the update loop. Register fire, smoke, and wood and a whole scene becomes buildable: pour sand into a wooden frame, drop one flame on it, wait for the wood to catch, then put it out with water and watch the smoke rise.

***Why the swap grid matters here.** Fire writes to its neighbours in  $G'$  and may itself move during the same tick. Both writes land in  $G'$ , so neither cell sees a change it caused. Which write wins is arbitrary, but at the scale of one tick it does not matter—the scene still plays out correctly on the next pass.*

---

## A NOTE ON GIT WORKFLOW

---

You have been using git since Week 1. This week it earns its keep. The template is a working program you are about to modify, and at least one change will break it. A branch means that when one does, you rewind to main and start the experiment over instead of untangling it by hand. Here is what to do, stated plainly.

Before writing any code, create a branch:

```
git checkout -b sand-physics
```

Work on that branch. When sand and water work, commit with a message that says what changed:

```
feat: implement sand fall and water spread rules

Sand falls straight down, slides diagonally when
blocked. Water does the same, plus spreads sideways
when it cannot fall at all.
```

Then merge back to main:

```
git checkout main
git merge sand-physics
```

If you want to experiment with fire or new materials, start another branch from main. The point is that “main” always contains something that works, and every experiment lives on its own branch where it cannot break the working version.

**Commit messages.** A good commit message names the change and says enough that you can reconstruct what you did a week later without reading the diff. “update” tells you nothing. “implement sand fall and water spread rules” tells you everything. One sentence for the summary, a short paragraph if the change is non-obvious.

---

## III. PUTTING IT TOGETHER

---

Get sand and water working first. Then—if you have time—add fire, smoke, and wood. Push everything to your repository before the due date.

DELIVERABLE. Your repository should contain:

- `assignment/temp.py` (or a renamed copy) with your physics implemented in `update()`.
- A `README.md` answering the two questions in the **aside** boxes, and a short paragraph on what you observed when running your simulation.
- A clean git history on `main` with descriptive commit messages.

**What to write in your brief.** In your `README.md`, answer the two questions in the aside boxes above and add a paragraph describing what you saw when you ran the simulation. Did sand pile up the way you expected? Did water pool flat? If you added bonus materials, describe the scene you built and what surprised you.